

Tesztelési Dokumentáció

Python I. beadandó feladat

Készítette: Majer Balázs (USUL2D)

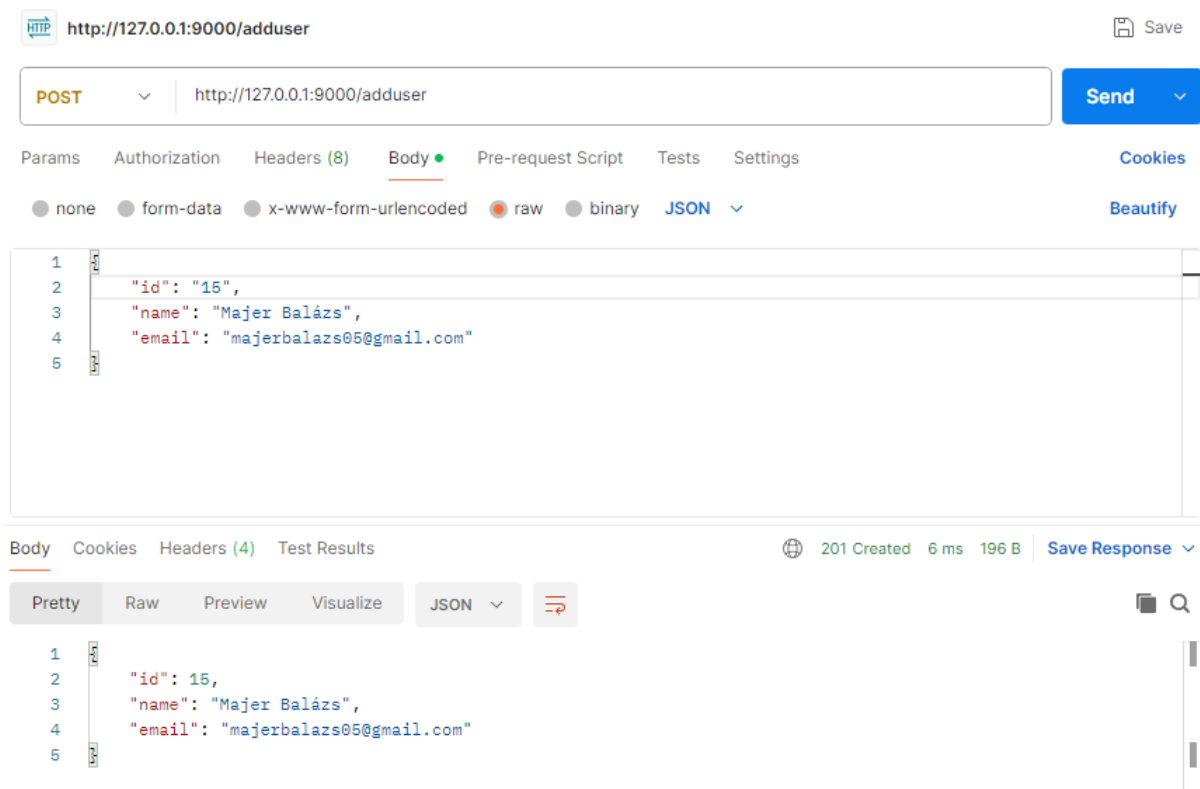
1. Tesztelési Környezet

A REST API tesztelése a fejlesztői környezetben, localhost-on (127.0.0.1:9000) történt. A végpontok ellenőrzéséhez a FastAPI beépített Swagger UI felületét (/docs), valamint a Postman nevű külső szoftvert használtam. A tesztek során ellenőriztem a sikeres működést és a megfelelő hibakezelést is.

2. Végpontok Tesztelése Postman segítségével

2.1. Új felhasználó hozzáadása (POST /adduser)

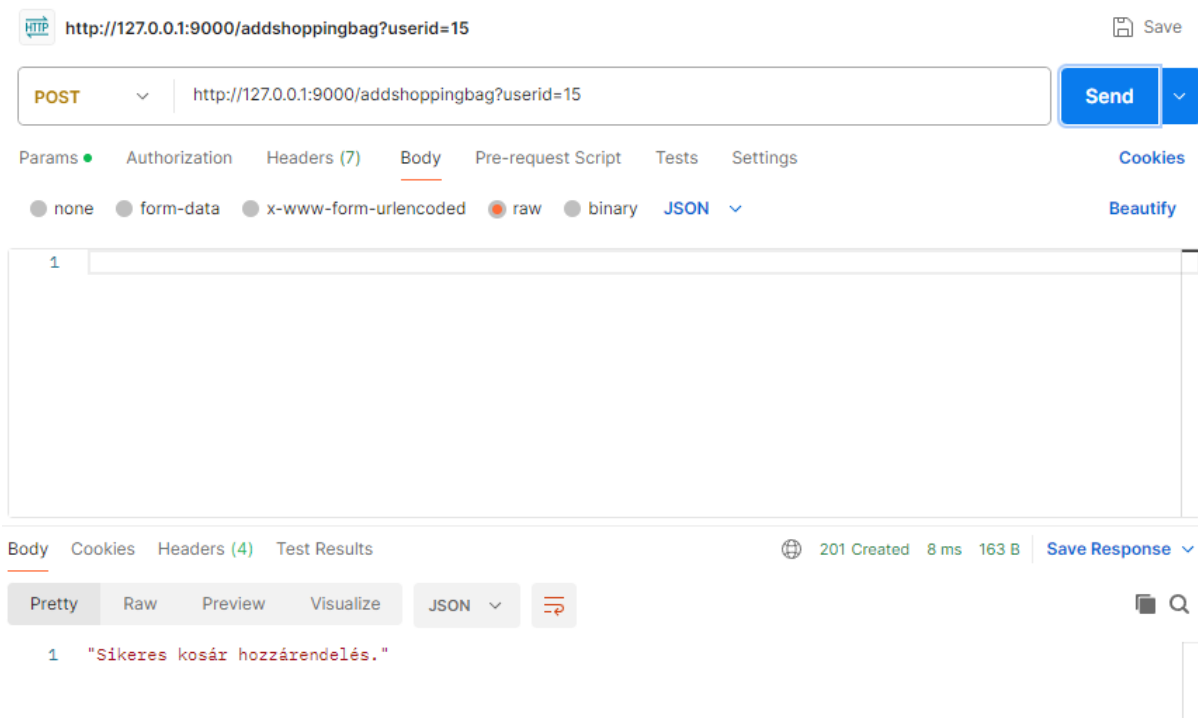
- **Cél:** Egy új felhasználó rögzítése az adatbázisban a megfelelő validációs szabályok figyelembevételével.
- **Bemenet:** JSON formátumú felhasználói adatok.
- **Eredmény:** A rendszer 201-es státuszkóddal válaszolt, és a felhasználó elmentésre került a data.json fájlba.



2.2. Kosár hozzárendelése (POST /addshoppingbag)

- **Cél:** Egy üres kosár létrehozása a korábban felvett felhasználóhoz.

- **Bemenet:** userid paraméter az URL-ben.
- **Eredmény:** A rendszer legenerálta a kosarat, 201-es kóddal és a "Sikeres kosár hozzárendelés." üzenettel válaszolt.



The screenshot shows a REST client interface with the following details:

- URL:** `http://127.0.0.1:9000/addshoppingbag?userid=15`
- Method:** POST
- Body:** (Empty)
- Response:**
 - Status: 201 Created
 - Time: 8 ms
 - Size: 163 B
 - Body: `"Sikeres kosár hozzárendelés."`

2.3. Termék hozzáadása a kosárhoz (POST /additem)

- **Cél:** Egy adott termék elhelyezése a felhasználó kosarában.
- **Bemenet:** userid paraméter és a termék adatai JSON formátumban.
- **Eredmény:** A rendszer frissítette a kosarat és visszatért annak új tartalmával.

The screenshot shows a REST client interface. At the top, the URL is `http://127.0.0.1:9000/additem?userid=15`. The method is set to **POST**. The request body is a JSON object:

```
1 {
2   "item_id": "0",
3   "name": "Gabonapehely",
4   "brand": "Nestle",
5   "price": "1500",
6   "quantity": "1"
7 }
```

The response status is **201 Created** with a response time of **7 ms** and a size of **245 B**. The response body is a JSON object:

```
1 {
2   "id": 107,
3   "user_id": 15,
4   "items": [
5     {
6       "item_id": 0,
7       "name": "Gabonapehely",
8       "brand": "Nestle",
9       "price": 1500.0,
10      "quantity": 1
11    }
12  ]
13 }
```

2.4. A kosár tartalmának és összértékének lekérdezése

- **Cél:** Ellenőrizni, hogy a végpontok megfelelően olvassák-e ki az adatokat a `data.json` fájlból.
- **A lekérdezett végpontok:** `GET /shoppingbag` és `GET /getusertotal`.
- **Eredmény:** A rendszer listázta a kosárban lévő termékeket, a `getusertotal` pedig kiszámolta a fizetendő összeget.

A feladatkiírás értelmében minden függvény tartalmaz hibakezelést (HTTPException). Ennek működését a beépített Swagger UI-on (/docs) teszteltem.

3.1. Validációs hiba (422 Unprocessable Entity)

- **Teszt:** Megpróbáltam egy új felhasználót rögzíteni hibás e-mail formátummal
- **Eredmény:** A Pydantic séma validátora megfogta a hibát, ValueError-t dobott, amit a szerver egy 422-as HTTP hibaüzenettel továbbított a kliensnek.

Code	Details
422	Error: Unprocessable Entity Response body <pre>{ "detail": [{ "type": "value_error", "loc": ["body", "email"], "msg": "Value error, Érvénytelen e-mail formátum!", "input": "bazz@webshop.hu", "ctx": { "error": {} }, "url": "https://errors.pydantic.dev/2.5/v/value_error" }] }</pre>

3.2. Nem létező adat lekérése (404 Not Found)

- **Teszt:** Megpróbáltam lekérdezni egy olyan felhasználó adatait (GET /user), aki nem létezik az adatbázisban.
- **Eredmény:** A rendszer a 404 - Not found státuszkóddal válaszolt.

404 <i>Undocumented</i>	Error: Not Found Response body <pre>{ "detail": "Nem található felhasználó a következő azonosítóval: 99" }</pre>
----------------------------	---